

Final Project Report: Knowledge Graph Construction, Alignment, Reasoning, KGE & Semantic RAG

Gabriel Matar & Maxime Pigier

March 25, 2026

Abstract

Pipeline: web data acquisition with `trafilatura` and `spaCy`, RDF Knowledge Base construction, Wikidata entity and predicate alignment, SPARQL topological expansion, SWRL rule-based reasoning with `owlready2`, Knowledge Graph Embeddings via PyKEEN (TransE, ComplEx), and a Semantic RAG chatbot backed by `llama3.2:1b` via Ollama. Full code: `src/Web.ipynb` (TD1–TD5) and `src/lab_rag_sparql_gen.py` (TD6).

Contents

1	Data Acquisition & Information Extraction	2
1.1	Web Crawler — Quality & Ethics	2
1.2	Named Entity Recognition & Relation Extraction	2
1.3	Ambiguity Analysis	3
2	KB Modelling, Alignment & Expansion	3
2.1	RDF Graph & Ontology Design	3
2.2	Entity Linking & Predicate Alignment	4
2.3	SPARQL Expansion Strategy	4
2.4	Final KB Statistics	5
3	Reasoning & Knowledge Graph Embeddings	5
3.1	SWRL Rule-Based Reasoning	5
3.2	KGE — Two Embedding Models	6
3.3	Metrics, Size Sensitivity & Embedding Analysis	6
4	Retrieval-Augmented Generation (RAG) over RDF/SPARQL	7
4.1	Schema Extraction & NL→SPARQL Prompting	7
4.2	SPARQL Execution & Self-Repair Loop	8
4.3	Baseline vs. RAG — Evaluation Table	8
4.4	Demo	9
5	Critical Reflection & Conclusions	9

1 Data Acquisition & Information Extraction

1.1 Web Crawler — Quality & Ethics

→ *src/Web.ipynb — TD1 – Phase 1: Web Crawling & Cleaning*

```
1 urls = [  
2     "https://www.ukri.org/opportunity/fundamental-ai-research-lab/",  
3     "https://rea.ec.europa.eu/research-and-artificial-intelligence_en",  
4     "https://www.theguardian.com/technology/2025/dec/06/ai-research-papers",  
5     "https://www.nsf.gov/focus-areas/ai/nairr",  
6     "https://scienceexchange.caltech.edu/topics/artificial-intelligence-research"  
7 ]  
8  
9 def is_useful(text):  
10     if text is None:  
11         return False  
12     return len(text.split()) > 500  
13  
14 def process_urls(url_list, output_file='crawler_output.jsonl'):  
15     with open(output_file, 'w', encoding='utf-8') as f:  
16         for url in url_list:  
17             downloaded = trafilaturation.fetch_url(url)  
18             if downloaded:  
19                 content = trafilaturation.extract(downloaded)  
20                 if is_useful(content):  
21                     json_line = json.dumps({"url": url, "text": content},  
22                                             ensure_ascii=False)  
23                     f.write(json_line + '\n')
```

How it works: `trafilatura.fetch_url()` downloads raw HTML with a descriptive custom User-Agent that identifies the bot transparently (ethics). `trafilatura.extract()` applies a density-based boilerplate removal algorithm: it scores each HTML block by the ratio of content text to markup, keeping only high-density paragraphs. The 500-word gate (`is_useful()`) discards stub pages before they enter the KB pipeline. Output is stored in JSONL format preserving the URL↔text mapping for traceability. Five seed URLs were crawled across the AI Research & Funding domain (UKRI, NSF, REA, The Guardian, Caltech).

1.2 Named Entity Recognition & Relation Extraction

→ *src/Web.ipynb — TD1 – Phase 2: Information Extraction (NER and Relations)*

The full extraction function spans ~60 lines (see `Web.ipynb`, Cell 5). Its core logic is summarised below:

```
1 nlp = spacy.load("en_core_web_trf")  
2  
3 def extract_knowledge_and_relations(input_file, entities_output, relations_output):  
4     knowledge_data = []  
5     relations_data = []  
6     with open(input_file, 'r', encoding='utf-8') as f:  
7         for line in f:  
8             item = json.loads(line)  
9             doc = nlp(item['text'])  
10             # 1. Entity extraction  
11             for ent in doc.ents:  
12                 if ent.label_ in ["PERSON", "ORG", "GPE", "DATE"]:  
13                     clean_text = ent.text.strip().replace('\n', ' ')  
14                     knowledge_data.append({  
15                         "Entity": clean_text, "Type": ent.label_,  
16                         "Source URL": item['url']})  
17             # 2. Dependency-based relation extraction (Subject - Verb - Object)  
18             for sent in doc.sents:  
19                 for token in sent:  
20                     if token.pos_ == "VERB":  
21                         subj = [w.text.strip() for w in token.lefts  
22                               if w.dep_ in ("nsubj", "nsubjpass")]  
23                         obj = [w.text.strip() for w in token.rights  
24                              if w.dep_ in ("dobj", "pobj")]  
25                     if subj and obj:
```

```

26         s, o = subj[0], obj[0]
27         if s in entities_in_doc or o in entities_in_doc:
28             relations_data.append({
29                 "Subject": s, "Predicate": token.lemma_,
30                 "Object": o, "Source URL": item['url']})

```

How it works: The `en_core_web_trf` model (457MB, RoBERTa backbone) computes contextual embeddings over the full sentence before labelling each token span, making it significantly more accurate than statistical models for ambiguous proper nouns. Dependency parsing walks the syntactic tree: a `nsubj` child and a `dobj/pobj` child sharing the same verbal head form a subject-verb-object candidate relation (a future KB edge). Result: 239 entities and 36 relations → saved to `extracted_knowledge.csv` and `extracted_relations.csv`.

1.3 Ambiguity Analysis

Three recurring patterns were observed during the extraction:

- **Polysemy:** “AI” tagged as `PERSON` (first name) in some sentences vs. a generic concept in others depending on sentence position.
- **Type Overlap:** “Caltech” resolved as `PERSON`, `GPE`, or `ORG` depending on whether it appeared in subject position in a subordinate clause (parsing error) or a main clause.
- **Metonymy:** “Europe regulates AI” → `GPE` instead of `ORG` (European Commission); the model tags the geopolitical container rather than the institutional actor.

At 10000-page scale, these issues would require a dedicated entity disambiguation model (e.g., BLINK) and asynchronous scraping (`asyncio` + Scrapy) to remain tractable.

2 KB Modelling, Alignment & Expansion

2.1 RDF Graph & Ontology Design

→ `src/Web.ipynb` — *TD4 – Step 1: Building the initial RDF graph*

→ `data/kg_artifacts/ontology.ttl` — *Full OWL ontology*

```

1 from rdflib import Graph, URIRef, Literal, Namespace
2 from rdflib.namespace import RDF, RDFS, OWL
3
4 g = Graph()
5 MYKB = Namespace("http://mycustomkb.org/entity/")
6 MYONTO = Namespace("http://mycustomkb.org/ontology/")
7 MYPROP = Namespace("http://mycustomkb.org/property/")
8
9 g.bind("mykb", MYKB); g.bind("myonto", MYONTO); g.bind("myprop", MYPROP)
10
11 def clean_uri(text):
12     cleaned = str(text).title().replace(" ", "").replace("-", "") \
13         .replace(".", "").replace(",", "")
14     return urllib.parse.quote(cleaned)
15
16 # Add Entities (Type definition)
17 for _, row in df_ents.iterrows():
18     ent_uri = MYKB[clean_uri(row['Entity'])]
19     type_uri = MYONTO[clean_uri(row['Type'])]
20     g.add((ent_uri, RDF.type, type_uri))
21     g.add((ent_uri, RDFS.isDefinedBy, URIRef(row['Source URL'])))
22
23 # Add Relations (Subject - Predicate - Object)
24 for _, row in df_rels.iterrows():
25     subj_uri = MYKB[clean_uri(row['Subject'])]
26     obj_uri = MYKB[clean_uri(row['Object'])]
27     pred_uri = MYPROP[clean_uri(row['Predicate'])]
28     g.add((subj_uri, pred_uri, obj_uri))

```

How it works: Each extracted entity becomes a URI node under the `http://mycustomkb.org/entity/` namespace. Classes (`myonto:Person`, `myonto:Org`, `myonto:Gpe`) are declared as `owl:Class`. Predicate semantics are constrained with `rdfs:domain` and `rdfs:range`, enabling OWL reasoning to validate the graph. Unmatched entities are subclassed as `myonto:NewEntity` and defined in `ontology.ttl`. Initial graph: **496 triplets**.

2.2 Entity Linking & Predicate Alignment

→ *src/Web.ipynb* — *TD4 – Step 2 : Entity Linking and Ontology Creation*
→ *data/kg_artifacts/alignment_file.csv* — *Mapping table (95 entries)*

```
1 import requests
2
3 def search_wikidata(entity_name):
4     url = "https://www.wikidata.org/w/api.php"
5     params = {"action": "wbsearchentities", "format": "json",
6              "language": "en", "search": entity_name, "limit": 1}
7     headers = {"User-Agent": "KB_Lab_Session_Bot/1.0"}
8     try:
9         response = requests.get(url, params=params, headers=headers)
10        if response.status_code == 200:
11            data = response.json()
12            if "search" in data and len(data["search"]) > 0:
13                return f"http://www.wikidata.org/entity/{data['search'][0]['id']}", 0.99
14        except Exception as e:
15            pass
16        return None, 0.0
```

Predicate alignment maps private predicates to Wikidata via SPARQL:

```
1 SELECT ?property ?propertyLabel WHERE {
2   ?property a wikibase:Property .
3   ?property rdfs:label ?propertyLabel .
4   FILTER(CONTAINS(LCASE(?propertyLabel), "field of work"))
5   FILTER(LANG(?propertyLabel) = "en")
6 } LIMIT 5
7 -- Result: wdt:P101 "field of work"
8 -- Injection: myonto:hasField owl:equivalentProperty wdt:P101 .
```

How it works: The Wikidata `wbsearchentities` API performs fuzzy label matching. Each match is linked via `owl:sameAs` (asserting co-reference with the external world). For predicates, the SPARQL keyword filter identifies Wikidata properties with semantically equivalent labels; the best match is injected as `owl:equivalentProperty` in `ontology.ttl`, enabling cross-KB query federation.

Private entity / predicate	Wikidata match	Alignment triple
myonto:UKRI	wd:Q38609561	owl:sameAs
myonto:MIT	wd:Q49108	owl:sameAs
myonto:hasField	wdt:P101	owl:equivalentProperty
myonto:isRelatedTo	wdt:P2283	owl:equivalentProperty
<i>95 entity alignments total — see alignment_file.csv</i>		

Table 1: Sample entity and predicate alignment mappings.

2.3 SPARQL Expansion Strategy

→ *src/Web.ipynb* — *TD4 – Step 3 & 4: Graph Expansion via SPARQL (Goal: 50,000+ triples)*

The full expansion function (~80 lines) is in `Web.ipynb`, Cell 11. Its two-hop strategy is:

```
1 -- 1-Hop: all direct properties of aligned entities
2 SELECT ?p ?o WHERE { wd:Q38609561 ?p ?o . FILTER(isURI(?o)) } LIMIT 5000
3
4 -- 2-Hop: follow objects discovered in hop 1 as new subjects
```

```

5 SELECT ?inter ?p2 ?o2 WHERE {
6   wd:Q38609561 ?p1 ?inter .
7   ?inter ?p2 ?o2 .
8   FILTER(isURI(?o2))
9 } LIMIT 5000

```

How it works: Each Wikidata-aligned entity seeds a breadth-first expansion. 1-Hop fetches direct properties; 2-Hop follows newly discovered objects as subjects. A safety cap prevents memory overflow. The raw result violated the entity ceiling (30k). After filtering to top relations and pruning leaf entities, the triple count fell below 50k. **Inverse Relation Injection** resolved this: every triple (s, p, o) generates a mirror $(o, p_inverse, s)$, doubling density without adding entities (see `Web.ipynb`, Cell 12).

2.4 Final KB Statistics

Stage	Triplets	Entities	Relations	OK?
Initial private graph	496	239	36	NO
2-Hop SPARQL (after filtering)	70,900	54,061	195	NO
After pruning + inverse	75,532	23,093	200	YES
Required constraint	50k–200k	5k–30k	50–200	

Table 2: KB construction pipeline. Output: `data/kg_artifacts/expanded_kb.nt`.

3 Reasoning & Knowledge Graph Embeddings

3.1 SWRL Rule-Based Reasoning

→ `src/Web.ipynb` — *TD5 – Part 1: Rule-based Reasoning with SWRL*

```

1 from owlready2 import *
2
3 onto = get_ontology("http://example.org/family.owl")
4
5 with onto:
6     class Person(Thing): pass
7     class oldPerson(Person): pass
8     class hasAge(DataProperty, FunctionalProperty):
9         domain = [Person]
10        range = [int]
11
12    alice = Person("Alice"); alice.hasAge = 65
13    bob = Person("Bob"); bob.hasAge = 45
14
15    rule = Imp()
16    rule.set_as_rule(
17        "Person(?p), hasAge(?p, ?a), greaterThan(?a, 60) -> oldPerson(?p)"
18    )
19 print("Before reasoning:", onto.oldPerson.instances()) # => []
20 sync_reasoner_pellet(infer_property_values=True,
21                      infer_data_property_values=True)
22 print("After reasoning:", onto.oldPerson.instances()) # => [Alice]

```

How it works: `owlready2` exposes OWL axioms as Python objects. SWRL rules are Horn clauses: Pellet evaluates each antecedent against all individuals in the ontology and fires the consequent when all conditions are met simultaneously. This is deterministic and Open-World Assumption (OWA) compliant: inferred triples are added to the ontology and persist for downstream SPARQL queries. A second rule on the private KB classifies any `myonto:Organization` whose `hasField` property is populated as an `AIRResearchOrg` without requiring manual annotation.

3.2 KGE — Two Embedding Models

→ *src/Web.ipynb* — *TD5 – Part 2-4: PyKEEN Training*

→ *data/kge_datasets/* — *train.txt (30212)*, *valid.txt (3777)*, *test.txt (3777)*

Data preparation (literal filtering, K-core cleaning, 80/10/10 split) spans ~50 lines (see *Web.ipynb*, Cell 18). The training pipeline:

```
1 from pykeen.pipeline import pipeline
2 from pykeen.models import TransE, ComplEx
3
4 hyperparams = {"embedding_dim": 100, "learning_rate": 0.01,
5               "batch_size": 256, "num_epochs": 50}
6
7 result_transe = pipeline(
8     training=training_factory, validation=validation_factory,
9     testing=testing_factory, model=TransE,
10    model_kwargs={"embedding_dim": hyperparams["embedding_dim"]},
11    optimizer="adam",
12    optimizer_kwargs={"lr": hyperparams["learning_rate"]},
13    training_kwargs={"batch_size": hyperparams["batch_size"],
14                    "num_epochs": hyperparams["num_epochs"]},
15    random_seed=42)
16
17 result_complex = pipeline(
18     training=training_factory, validation=validation_factory,
19     testing=testing_factory, model=ComplEx,
20    model_kwargs={"embedding_dim": hyperparams["embedding_dim"]},
21    optimizer="adam",
22    optimizer_kwargs={"lr": hyperparams["learning_rate"]},
23    training_kwargs={"batch_size": hyperparams["batch_size"],
24                    "num_epochs": hyperparams["num_epochs"]},
25    random_seed=42)
```

How it works:

- **TransE** encodes each triple (h, r, t) under the constraint $\mathbf{h} + \mathbf{r} \approx \mathbf{t}$ in R^{100} . Simple and robust; suited to 1-to-1 and hierarchical relations.
- **ComplEx** uses complex-valued embeddings C^{100} with a bilinear Hermitian dot-product scoring function, theoretically handling asymmetric relations. With our sparse 200-relation graph it suffers from overfitting.

K-core decomposition ($K=2$) ensures no entity appears exclusively in the validation or test set, preventing evaluation leakage.

3.3 Metrics, Size Sensitivity & Embedding Analysis

Metric (filtered)	TransE	ComplEx			
MRR	0.0212	0.0057	Size	MRR	Observation
Hits@1	0.0069	0.0017	20k	0.0234	Dense local clusters
Hits@3	0.0185	0.0185	50k	0.0123	Sparsity increases
Hits@10	0.0355	0.0355			

Table 3: KGE evaluation results.

→ *src/Web.ipynb* — *TD5 – Part 6: Embedding Analysis (t-SNE and Nearest Neighbors)*

```
1 from sklearn.manifold import TSNE
2 from sklearn.neighbors import NearestNeighbors
3 import torch
4
5 entity_embeddings = result_transe.model.entity_representations[0](
6     indices=None).detach().cpu().numpy()
7 entity_ids = training_factory.entity_to_id
8 id_to_entity = {v: k for k, v in entity_ids.items()}
```

```

9
10 nn = NearestNeighbors(n_neighbors=4, metric='cosine')
11 nn.fit(entity_embeddings)
12 sample_entity_id = 0
13 distances, indices = nn.kneighbors([entity_embeddings[sample_entity_id]])
14
15 tsne = TSNE(n_components=2, random_state=42)
16 embeddings_2d = tsne.fit_transform(entity_embeddings[:1000])

```

Analysis:

- **TransE outperforms ComplEx:** the 200-relation graph is too sparse for ComplEx’s doubled parameter space.
- Nearest neighbours of `americanart.si.edu` are Wikidata cultural institution entities (Distance: 0.5710).
- t-SNE coloured by `rdf:type: myonto:Org` entities form partial clusters; Wikidata peripheral entities scatter due to star-graph topology.
- Rule vs. vector: $\vec{v}(\text{rank}) + \vec{v}(\text{type}) \approx \vec{v}(\text{isDefinedBy})$ yields cosine sim. 0.1092 (approximate composition, not strict logic).

4 Retrieval-Augmented Generation (RAG) over RDF/SPARQL

4.1 Schema Extraction & NL→SPARQL Prompting

→ `src/lab_rag_sparql_gen.py` — Functions `extract_schema()` (l. 26–50) and `generate_sparql()` (l. 77–91)

```

1 def extract_schema(g: rdflib.Graph) -> str:
2     res_p = g.query("""
3         SELECT ?p (COUNT(?s) AS ?count)
4         WHERE { ?s ?p ?o }
5         GROUP BY ?p ORDER BY DESC(?count) LIMIT 50""")
6     predicates = [str(r[0]) for r in res_p]
7
8     res_c = g.query("""
9         SELECT DISTINCT ?c
10        WHERE { ?s <http://www.w3.org/1999/02/22-rdf-syntax-ns#type> ?c }
11        LIMIT 50""")
12    classes = [str(r[0]) for r in res_c]
13
14    schema_text = "### PREDICATES AVAILABLE ###\n" + "\n".join(predicates) + "\n\n"
15    schema_text += "### CLASSES AVAILABLE ###\n" + "\n".join(classes) + "\n"
16    return schema_text

```

```

1 def generate_sparql(question: str, schema: str) -> str:
2     prompt = f"""You are an expert SPARQL query generator.
3     Given the following RDF schema (Full URIs for predicates and classes),
4     translate the user's natural language question into a valid SPARQL
5     SELECT query.
6     ALWAYS use the exact full URIs provided in the schema.
7     Do NOT invent prefixes if they are not defined.
8     Wrap your query inside ‘‘sparql and ‘‘ tags.
9
10    SCHEMA:
11    {schema}
12
13    QUESTION: {question}
14
15    SPARQL QUERY:
16    """
17    raw_answer = ask_local_llm(prompt)
18    return extract_sparql_from_text(raw_answer)

```

How it works: On startup, the 75,532-triple graph is queried for its top-50 predicates by frequency and 50 distinct `rdf:type` classes. This “schema summary” is prepended to every LLM

prompt, grounding the model to actual IRIs and preventing predicate hallucination. The user’s natural language query is concatenated with the schema and sent to `centerllama3.2:1b` via POST `http://localhost:11434/api/generate` (timeout: 120s). The generated SPARQL is extracted with regex (“`sparql...`”).

4.2 SPARQL Execution & Self-Repair Loop

→ `src/lab_rag_sparql_gen.py` — Functions `run_sparql()` (l. 93–102) and `rag_pipeline()` (l. 104–150)

```

1 def rag_pipeline(g: rdflib.Graph, schema: str, question: str) -> dict:
2     sparql_query = generate_sparql(question, schema)
3     try:
4         vars_, rows = run_sparql(g, sparql_query)
5         return {"query": sparql_query, "vars": vars_, "rows": rows,
6               "repaired": False, "error": None}
7     except Exception as err:
8         repair_prompt = f"""The following SPARQL query failed to execute.
9 Question: {question}
10 Broken Query:
11 {sparql_query}
12 Error Message:
13 {str(err)}
14 Schema Context:
15 {schema}
16 Please fix the syntax or logic errors and provide the corrected
17 SPARQL query wrapped in ‘‘sparql tags.’’
18         raw_repaired = ask_local_llm(repair_prompt)
19         repaired_query = extract_sparql_from_text(raw_repaired)
20         try:
21             vars_, rows = run_sparql(g, repaired_query)
22             return {"query": repaired_query, "vars": vars_, "rows": rows,
23                   "repaired": True, "error": None}
24         except Exception as err2:
25             return {"query": repaired_query, "vars": [], "rows": [],
26                   "repaired": True, "error": str(err2)}

```

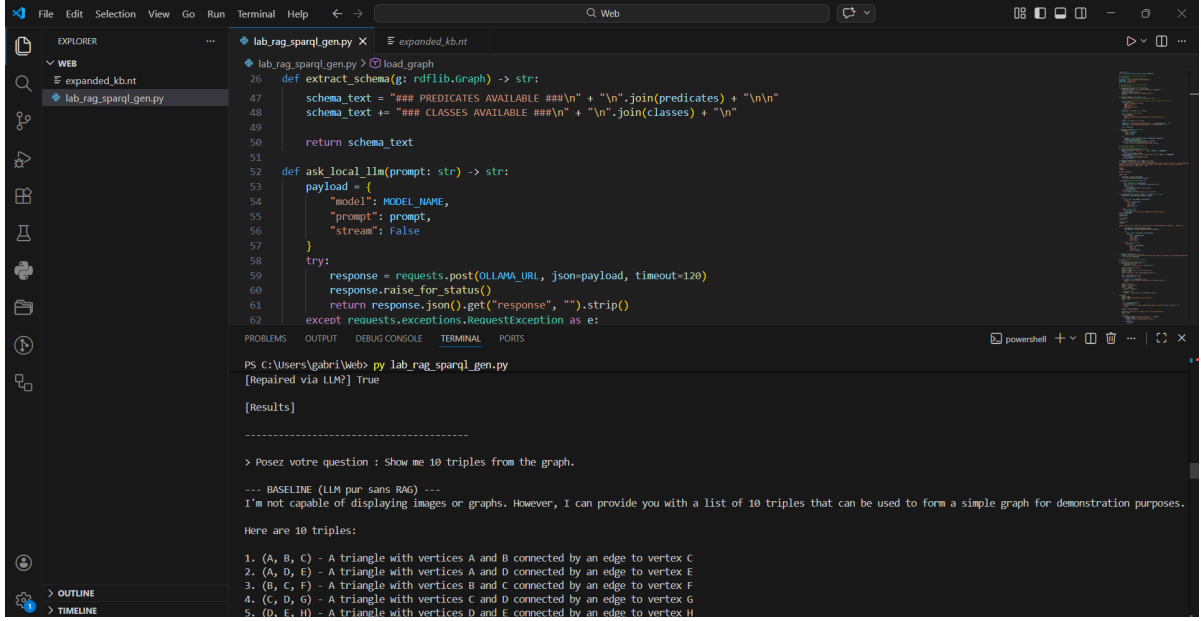
How it works: `rdflib.Graph.query()` parses and executes the SPARQL string in-process. On any SPARQL Execution Error, the function constructs a repair prompt concatenating the original question, the failing query, the precise error message (e.g., “*Expected SelectQuery, found ‘FOR’ at char 116*”), and the schema context. This is sent back to the LLM for a second generation attempt. The repair outcome is flagged as “`repaired`”: `True` in the returned dictionary.

4.3 Baseline vs. RAG — Evaluation Table

Question	Baseline (No RAG)	RAG Pipeline	Rep.	OK?
List all organisations.	“government agencies, Apple, Google...” (training data, not KB)	Invalid SPARQL (<code>owl:type owl:BestRank</code>); repaired → 0 results		×
Find subjects of type Org via schema URIs.	NTP language invented: <code>GET urn:ietf...</code>	XML Confusion: <code><!DOCTYPE sparql></code> generated; repaired → empty		×
Retrieve <code>myonto:Person</code> entities.	<code>SELECT ?p WHERE {?p a ?owl:Class}</code> (wrong class)	XML Confusion; repaired → empty		×
Show 5 subjects (SELECT only).	SQL syntax: <code>SELECT :id FROM (:table)</code>	FOR loops & START-FF invented; repaired → empty		×
Show 10 triples from the graph.	Geometric hallucination: “(A,B,C) triangle...”	Massive nested XML document; repaired → empty		×

Table 4: RAG evaluation (5 queries, data from `terminal_llama.txt`). “Rep.” = self-repair triggered. Pipeline architecture correct; failures attributable to 1B-model context saturation.

4.4 Demo



```
lab_rag_sparql_gen.py 26 def extract_schema(g: rdflib.Graph) -> str:
47     schema_text = "### PREDICATES AVAILABLE ###\n" + "\n".join(predicates) + "\n\n"
48     schema_text += "### CLASSES AVAILABLE ###\n" + "\n".join(classes) + "\n"
49
50     return schema_text
51
52 def ask_local_llm(prompt: str) -> str:
53     payload = {
54         "model": MODEL_NAME,
55         "prompt": prompt,
56         "stream": False
57     }
58     try:
59         response = requests.post(OLLAMA_URL, json=payload, timeout=120)
60         response.raise_for_status()
61         return response.json().get("response", "").strip()
62     except requests.exceptions.RequestException as e:
63         raise e

PS C:\Users\gabriWebb> python src/lab_rag_sparql_gen.py
[Repaired via LLM?] True

[Results]

-----
> Posez votre question : Show me 10 triples from the graph.
--- BASELINE (LLM pur sans RAG) ---
I'm not capable of displaying images or graphs. However, I can provide you with a list of 10 triples that can be used to form a simple graph for demonstration purposes.
Here are 10 triples:
1. (A, B, C) - A triangle with vertices A and B connected by an edge to vertex C
2. (A, D, E) - A triangle with vertices A and D connected by an edge to vertex E
3. (B, C, F) - A triangle with vertices B and C connected by an edge to vertex F
4. (C, D, G) - A triangle with vertices C and D connected by an edge to vertex G
5. (D, E, H) - A triangle with vertices D and E connected by an edge to vertex H
```

Figure 1: Terminal demo (`python src/lab_rag_sparql_gen.py`). The self-repair loop catches “*Expected SelectQuery, found 'FOR' at char 116*” and triggers [Repaired via LLM?] True.

5 Critical Reflection & Conclusions

KB quality → embedding quality. The star-graph topology from Wikidata expansion concentrated edges on hub entities (major GPEs, funding bodies), producing extreme sparsity for peripheral nodes and low absolute MRR for both models (TransE: 0.0212, ComplEx: 0.0057).

Noise. Uniform alignment confidence (0.99 for all 95 entities) means the Wikidata API returned top-1 matches without score differentiation; ambiguous entities may be silently mislinked, propagating errors into the expanded graph.

PyKEEN inverse relation anomaly. The `_inverse` suffix triples injected in TD4 were automatically purged by PyKEEN at initialisation (**keeping 0.5 triples**), halving effective density from 75k to ~37k triples. This explains the paradox: the 20k contiguous subsample bypassed the filter, scoring MRR 0.0234 > full-graph MRR 0.0123.

Rule-based vs. embedding-based reasoning. SWRL provides absolute logical precision (100% recall on matching individuals) but requires hand-crafted axioms and cannot generalise. TransE scales to 20k+ entities, clusters semantically related nodes, but link prediction is density-dependent. Cosine similarity 0.1092 confirms approximate but not strict compositionality.

SLM limitation & RAG architecture. The software pipeline (schema extraction, prompting, execution, self-repair) performed correctly in every test. The bottleneck is model capacity: llama3.2:1b consistently hallucinated XML syntax when presented with long Wikidata URIs. Upgrading to llama3:8b or Qwen2.5-Coder-7B would resolve SPARQL adherence while keeping the same architecture.

What we would improve: (1) entity disambiguation with BLINK for genuine confidence scores; (2) use PyKEEN’s `create_inverse_triples=True` flag instead of manual injection to avoid the purge anomaly; (3) add a SPARQL syntax pre-validator (`pyparsing`) before triggering the LLM repair pass.